Singly Linked List

What is a Singly Linked List?

A **Singly Linked List** is a **linear data structure** in which elements are stored in **nodes**, and each node contains a **pointer that points to the next node in the sequence**.

Unlike arrays, the elements of a singly linked list are **not stored in contiguous memory locations**. Instead, nodes are connected through **links (pointers)**.

Each node contains two parts:

1. **Data field** – stores the actual value
2. **Next field (Pointer)** – stores the address of the next node

The address of first node in the list is stored in the **Head pointer**, and the last node points to **NULL**, which indicates the end of the list.

Structure of a Node

A node in a singly linked list consists of two components:

- **Data** – the value stored in the node
- **Next Pointer** – reference to the next node

Representation:

```
[ Data | Next ]
```

Example linked list:

```
Head → [10 | • ] → [20 | • ] → [30 | NULL]
```

Declaration of Singly Linked List Node in C

The node structure is declared using a **structure** in C.

```
struct Node {
   int data;
   struct Node* next;
};
```

Explanation:

- `struct Node` defines the node type.
- `int data` stores the value of the node.
- `struct Node* next` is a pointer that stores the address of the next node.

To create a pointer to the head node:

```
struct Node* head;
```

Here, `head` points to the **first node of the linked list**.

Characteristics of Singly Linked List

- Nodes are **dynamically allocated in memory**.
- Each node stores the **address of the next node**.
- Traversal can only be done **in one direction**.
- The list starts from the **head node**.
- The last node contains **NULL in the next pointer**.

Advantages of Singly Linked List

1. **Dynamic Size**

   The size of the list can grow or shrink during program execution. Unlike arrays, it does not require a fixed size.

2. **Efficient Insertion and Deletion**

   Inserting or deleting elements does not require shifting elements as in arrays.

3. **Better Memory Utilization**

   Memory is allocated only when needed.

4. **Flexible Data Structure**

   Linked lists can easily expand without needing continuous memory.

5. **Useful for Dynamic Applications**

   They are suitable when the number of elements changes frequently.

Disadvantages of Singly Linked List

1. **Extra Memory Usage**

   Each node requires additional memory for the pointer.

2. **No Random Access**

   Elements cannot be accessed directly by index like arrays.

3. **Traversal is Required**

   To access a specific element, we must traverse the list from the beginning.

4. **Only One Direction Traversal**

   Movement is possible only forward.

Real-Life Use Cases of Singly Linked List

Singly linked lists are used in many real-world applications:

1. Music Playlist

Music players store songs in a linked structure where each song points to the **next song** in the playlist.

2. Navigation Systems

Navigation apps maintain a sequence of locations where each location links to the **next destination**.

3. Undo/Redo Operations

Applications store recent actions in linked structures so they can be reversed.

4. Browser History

Web browsers maintain visited pages in order where each page links to the next.

5. Memory Management

Operating systems use linked lists to manage **free memory blocks**.

6. Implementing Other Data Structures

Linked lists are used to implement:

- Stacks
- Queues
- Graph adjacency lists

Key Points to Remember

- A singly linked list consists of **nodes connected using pointers**.
- Each node contains **data and a next pointer**.
- The first node address is stored in **head**.
- The last node points to **NULL**.
- Traversal is possible **only in one direction**.